Proyecto 1

Generado por Doxygen 1.16.1

Capítulo 1

Índice jerárquico

1.1. Jerarquía de clases

Este listado de herencia está ordenado de forma general pero no está en orden alfabético estricto:

CameraTransform . . . . . ??
Coord . . . . . ??
Extent . . . . . ??
Fragment . . . . . ??
Image . . . . . ??
Kernel . . . . . ??
Light . . . . . ??
Material . . . . . ??
Payload . . . . . ??
Pipeline . . . . . ??
QDockWidget
  CameraDock . . . . . ??
  LightDock . . . . . ??
  MaterialDock . . . . . ??
  ObjectDock . . . . . ??
QMainWindow
  Window . . . . . ??
QWidget
  Canvas . . . . . ??
Rect . . . . . ??
Sampler . . . . . ??
SamplerCube . . . . . ??
Transform . . . . . ??
Vertex . . . . . ??

Capítulo 2

Índice de clases

2.1. Lista de clases

Lista de clases, estructuras, uniones e interfaces con breves descripciones:

CameraDock
Puerto para el control de la cámara . . . . . ??
CameraTransform
Transformación de una cámara . . . . . ??
Canvas
Lienzo donde se muestra la imagen en la ventana . . . . . ??
Coord
Coordenadas de un pixel en una imagen . . . . . ??
Extent
Tamaño de una región rectangular en una imagen . . . . . ??
Fragment
Fragmento generado por el rasterizador . . . . . ??
Image
Imagen con información del tamaño . . . . . ??
Kernel
Kernel simétrico para operaciones de convolución . . . . . ??
Light
Propiedades de una luz puntual . . . . . ??
LightDock
Puerto para el control de las luces . . . . . ??
Material
Propiedades de un material . . . . . ??
MaterialDock
Puerto para el control del material . . . . . ??
ObjectDock
Puerto para el control del objeto . . . . . ??
Payload
Carga que se transfiere entre etapas de la línea de renderizado . . . . . ??
Pipeline
Línea de renderizado por rasterización . . . . . ??
Rect
Región rectangular de pixeles en una imagen . . . . . ??
Sampler
Muestreado de una imagen 2D . . . . . ??
SamplerCube
Muestreado de un cubo . . . . . ??

Transform
Transformación de un objeto . . . . . ??
Vertex
Vértice que se procesa en la línea de renderizado . . . . . ??
Window
Ventana principal de la aplicación . . . . . ??

Capítulo 3

Índice de archivos

3.1. Lista de archivos

Lista de todos los archivos documentados y con breves descripciones:

include/app.hpp . . . . . ??
include/dip.hpp . . . . . ??
include/gmath.hpp . . . . . ??
include/gui.hpp . . . . . ??
include/pipeline.hpp . . . . . ??
include/sampler.hpp . . . . . ??

Capítulo 4

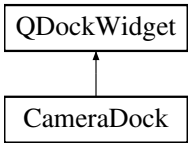
Documentación de clases

4.1. Referencia de la estructura CameraDock

Puerto para el control de la cámara.

`#include <gui.hpp>`

Diagrama de herencia de CameraDock



Métodos públicos

- **CameraDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QVBoxLayout **layout**
- QComboBox **camera_dropdown**
- QLabel **transform_label**
- QWidget **group**
- QGridLayout **group_layout**
- QLabel **row_labels** [2]
- QLabel **col_labels** [3]
- QDoubleSpinBox **pos_boxes** [3]
- QDoubleSpinBox **rot_boxes** [3]
- QPushButton **button**
- QLabel **proj_label**
- QRadioButton **proj_buttons** [2]
- QLabel **skybox_label**
- QComboBox **skybox_dropdown**

4.1.1. Descripción detallada

Puerto para el control de la cámara.

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.2. Referencia de la estructura CameraTransform

Transformación de una cámara.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **rotation** [4]

4.2.1. Descripción detallada

Transformación de una cámara.

La documentación de esta estructura está generada del siguiente archivo:

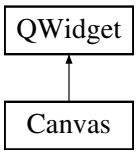
- include/app.hpp

4.3. Referencia de la estructura Canvas

Lienzo donde se muestra la imagen en la ventana.

```
#include <gui.hpp>
```

Diagrama de herencia de Canvas



Métodos públicos

- void paintEvent (QPaintEvent *event)

  Respuesta a un evento de redibujar el lienzo.

4.3.1. Descripción detallada

Lienzo donde se muestra la imagen en la ventana.

4.3.2. Documentación de funciones miembro

4.3.2.1. paintEvent()

```
void Canvas::paintEvent (
            QPaintEvent * event)
```

Respuesta a un evento de redibujar el lienzo.

Parámetros

event	el evento recibido

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.4. Referencia de la estructura Coord

Coordenadas de un pixel en una imagen.

```
#include <dip.hpp>
```

Atributos públicos

- uint64_t **x**
- uint64_t **y**

4.4.1. Descripción detallada

Coordenadas de un pixel en una imagen.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.5. Referencia de la estructura Extent

Tamaño de una región rectangular en una imagen.

```
#include <dip.hpp>
```

Atributos públicos

- uint64_t **width**
- uint64_t **height**

4.5.1. Descripción detallada

Tamaño de una región rectangular en una imagen.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.6. Referencia de la estructura Fragment

Fragmento generado por el rasterizador.

```
#include <pipeline.hpp>
```

Atributos públicos

- uint64_t **x**
- uint64_t **y**
- double **u**
- double **v**
- double **depth**
- double **color** [4]

4.6.1. Descripción detallada

Fragmento generado por el rasterizador.

La documentación de esta estructura está generada del siguiente archivo:

- include/pipeline.hpp

4.7. Referencia de la estructura Image

Imagen con información del tamaño.

```
#include <dip.hpp>
```

Atributos públicos

- Extent **extent**
- uint64_t **channels**
- void * **data**

4.7.1. Descripción detallada

Imagen con información del tamaño.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.8. Referencia de la estructura Kernel

Kernel simétrico para operaciones de convolución.

```
#include <dip.hpp>
```

Atributos públicos

- uint64_t **radius**
- void * **data**

4.8.1. Descripción detallada

Kernel simétrico para operaciones de convolución.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.9. Referencia de la estructura Light

Propiedades de una luz puntual.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **ambient** [4]
- double **diffuse** [4]
- double **specular** [4]

4.9.1. Descripción detallada

Propiedades de una luz puntual.

La documentación de esta estructura está generada del siguiente archivo:

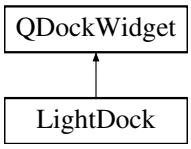
- include/app.hpp

4.10. Referencia de la estructura LightDock

Puerto para el control de las luces.

```
#include <gui.hpp>
```

Diagrama de herencia de LightDock



Métodos públicos

- **LightDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QVBoxLayout **layout**
- QComboBox **dropdown**
- QCheckBox **checkbox**
- QLabel **transform_label**
- QWidget **transform_group**
- QGridLayout **transform_group_layout**
- QLabel **transform_row_label**
- QLabel **transform_col_labels** [3]
- QDoubleSpinBox **pos_boxes** [3]
- QLabel **component_label**
- QWidget **component_group**
- QGridLayout **component_group_layout**
- QLabel **component_row_labels** [3]
- QLabel **component_col_labels** [4]
- QDoubleSpinBox **amb_boxes** [4]
- QDoubleSpinBox **diff_boxes** [4]
- QDoubleSpinBox **spec_boxes** [4]

4.10.1. Descripción detallada

Puerto para el control de las luces.

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.11. Referencia de la estructura Material

Propiedades de un material.

```
#include <app.hpp>
```

Atributos públicos

- double **ambient** [4]
- double **diffuse** [4]
- double **specular** [4]
- double **shininess**

4.11.1. Descripción detallada

Propiedades de un material.

La documentación de esta estructura está generada del siguiente archivo:

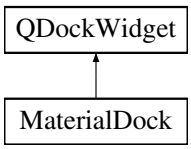
- include/app.hpp

4.12. Referencia de la estructura MaterialDock

Puerto para el control del material.

```
#include <gui.hpp>
```

Diagrama de herencia de MaterialDock



Métodos públicos

- **MaterialDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QVBoxLayout **layout**
- QComboBox **material_dropdown**
- QLabel **coef_label**
- QWidget **group**
- QGridLayout **group_layout**
- QLabel **row_labels** [3]
- QLabel **col_labels** [4]
- QDoubleSpinBox **amb_boxes** [4]
- QDoubleSpinBox **diff_boxes** [4]
- QDoubleSpinBox **spec_boxes** [4]
- QLabel **sh_label**
- QDoubleSpinBox **sh_box**
- QLabel **texture_label**
- QComboBox **texture_dropdown**
- QCheckBox **checkbox**

4.12.1. Descripción detallada

Puerto para el control del material.

La documentación de esta estructura está generada del siguiente archivo:

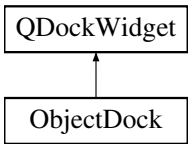
- include/gui.hpp

4.13. Referencia de la estructura ObjectDock

Puerto para el control del objeto.

```
#include <gui.hpp>
```

Diagrama de herencia de ObjectDock



Métodos públicos

- **ObjectDock** ()

  Construye un nuevo puerto.
- void **syncValues** ()

  Sincroniza los valores del puerto con los valores de las variables de la aplicación.

Atributos públicos

- QWidget **widget**
- QVBoxLayout **layout**
- QComboBox **dropdown**
- QLabel **label**
- QWidget **group**
- QGridLayout **group_layout**
- QLabel **row_labels** [3]
- QLabel **col_labels** [3]
- QDoubleSpinBox **pos_boxes** [3]
- QDoubleSpinBox **rot_boxes** [3]
- QDoubleSpinBox **scale_boxes** [3]

4.13.1. Descripción detallada

Puerto para el control del objeto.

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

4.14. Referencia de la estructura Payload

Carga que se transfiere entre etapas de la línea de renderizado.

```
#include <app.hpp>
```

Atributos públicos

- double **view** [3]
- double **normal** [3]
- double **light** [NUM_LIGHTS][3]
- double **texture** [2]

4.14.1. Descripción detallada

Carga que se transfiere entre etapas de la línea de renderizado.

La documentación de esta estructura está generada del siguiente archivo:

- include/app.hpp

4.15. Referencia de la estructura Pipeline

Línea de renderizado por rasterización.

```
#include <pipeline.hpp>
```

Métodos públicos

- void **clear_buffers** ()

  Limpia los buffers de color y profundidad.
- void **draw** ()

  Dibuja triángulos.
- bool clipping_test ()

  Realiza la prueba de recorte.
- void **to_plane** ()

  Pasa la primitiva actual al plano de proyección.
- void **to_viewport** ()

  Pasa la primitiva actual a la región de dibujo en la imagen.
- void **rasterize** ()

  Rasteriza la primitiva actual.
- bool depth_test ()

  Realiza la prueba de profundidad.
- void **map_barycentric_coordinates** ()

  @biref Transforma las coordenadas baricéntricas de un fragmento en el plano a sus coordenadas baricéntricas en el espacio antes de la proyección
- void **build_payload** ()

  Construye la carga del fragmento actual.
- void **blend_fragment** ()

  Asigna el color del fragmento al buffer de color.

Atributos públicos

- uint64_t **num_primitives**
- uint64_t **width**
- uint64_t **height**
- uint64_t **primitive_id**
- uint64_t **vertex_id**
- Fragment **fragment**
- Payload **out** [3]
- Payload **in**
- uint64_t **viewport** [2][2]
- uint64_t **raster_rect** [2][2]
- double **homogeneous_coordinates** [3]
- double **positions_3d** [3][3]
- double **boundary_3d** [2][3]
- double **positions_2d** [3][2]
- double **boundary_2d** [2][2]
- void ∗ **color_buffer**
- void ∗ **depth_buffer**
- void ∗ **vertex_buffer**
- Payload(∗ **vertex_shader**)(Vertex, double ∗)
- void(∗ **fragment_shader**)(Payload, double ∗)
- bool **front_face_clockwise**
- bool **invert_y_viewport**
- bool **depth_write_disabled**

4.15.1. Descripción detallada

Línea de renderizado por rasterización.

4.15.2. Documentación de funciones miembro

4.15.2.1. clipping_test()

```
bool Pipeline::clipping_test ()
```

Realiza la prueba de recorte.

Devuelve

`true` si y solo si la primitiva actual está en el volumen de visión

4.15.2.2. depth_test()

```
bool Pipeline::depth_test ()
```

Realiza la prueba de profundidad.

Devuelve

`true` si y solo si el fragmento actual es visible dada su profundidad

La documentación de esta estructura está generada del siguiente archivo:

- include/pipeline.hpp

4.16. Referencia de la estructura Rect

Región rectangular de pixeles en una imagen.

```
#include <dip.hpp>
```

Atributos públicos

- Coord **start**
- Coord **end**

4.16.1. Descripción detallada

Región rectangular de pixeles en una imagen.

La documentación de esta estructura está generada del siguiente archivo:

- include/dip.hpp

4.17. Referencia de la estructura Sampler

Muestreado de una imagen 2D.

`#include <sampler.hpp>`

Métodos públicos

- void raw_sample_nearest (double *position, double *sample)

  Realiza un muestreo directo de la imagen con el filtro de vecinos más cercanos.
- void raw_sample_bilinear (double *position, double *sample)

  Realiza un muestreo directo de la imagen con el filtro de interpolación bilineal.
- void sample (double *position, double *sample)

  Realiza un muestreo normalizado de la imagen.

Atributos públicos

- Image **image**
- SamplerFilter **filter**
- SamplerAddressMode **address_mode**

4.17.1. Descripción detallada

Muestreado de una imagen 2D.

4.17.2. Documentación de funciones miembro

4.17.2.1. raw_sample_bilinear()

```
void Sampler::raw_sample_bilinear (
            double * position,
            double * sample)
```

Realiza un muestreo directo de la imagen con el filtro de interpolación bilineal.

Parámetros

position	es la posición dentro de la imagen donde se va a muestrear. `position [0]` debe estar en el rango `[0, width - 1]`.`position [1]` debe estar en el rango `[0, height - 1]`.
sample	es donde se guardan los valores directos del muestreo. Los valores escritos en `sample` están en el rangpo `[0,255]`.

4.17.2.2. raw_sample_nearest()

```
void Sampler::raw_sample_nearest (
            double * position,
            double * sample)
```

Realiza un muestreo directo de la imagen con el filtro de vecinos más cercanos.

Parámetros

position	es la posición dentro de la imagen donde se va a muestrear. `position [0]` debe estar en el rango `[0, width - 1]` y `position [1]` en `[0, height - 1]`.

sample	es donde se guardan los valores directos del muestreo, en el rango `[0,255]`

4.17.2.3. sample()

```
void Sampler::sample (
            double * position,
            double * sample)
```

Realiza un muestreo normalizado de la imagen.

Parámetros

position	es la posición normalizada donde se va a muestrear. Los valores en `position` pueden estar fuera del rango normalizado `[0,1]`.
sample	es donde se guardan los valores del muestreo. Los valores escritos en `sample` están en el rangpo `[0,1]`.

La documentación de esta estructura está generada del siguiente archivo:

- include/sampler.hpp

4.18. Referencia de la estructura SamplerCube

Muestreado de un cubo.

`#include <sampler.hpp>`

Métodos públicos

- void sample (double ∗direction, double ∗sample)

  Realiza un muestreo normalizado del cubo.

Atributos públicos

- Image **cube_map** [6]
- SamplerFilter **filter**

4.18.1. Descripción detallada

Muestreado de un cubo.

4.18.2. Documentación de funciones miembro

4.18.2.1. sample()

```
void SamplerCube::sample (
            double * direction,
            double * sample)
```

Realiza un muestreo normalizado del cubo.

Parámetros

direction	es una vector de dirección en 3D, distinto del origen

sample	es donde se guardan los valores del muestreo. Los valores escritos en `sample` están en el rangpo `[0,1]`.

La documentación de esta estructura está generada del siguiente archivo:

- include/sampler.hpp

4.19. Referencia de la estructura Transform

Transformación de un objeto.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **rotation** [4]
- double **scale** [3]

4.19.1. Descripción detallada

Transformación de un objeto.

La documentación de esta estructura está generada del siguiente archivo:

- include/app.hpp

4.20. Referencia de la estructura Vertex

Vértice que se procesa en la línea de renderizado.

```
#include <app.hpp>
```

Atributos públicos

- double **position** [3]
- double **normal** [3]
- double **texture** [2]

4.20.1. Descripción detallada

Vértice que se procesa en la línea de renderizado.

La documentación de esta estructura está generada del siguiente archivo:

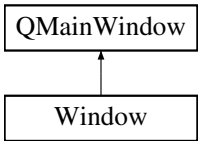
- include/app.hpp

4.21. Referencia de la estructura Window

Ventana principal de la aplicación.

```
#include <gui.hpp>
```

Diagrama de herencia de Window



Métodos públicos

- **Window** ()

  Construye una nueva ventana.
- void **draw** ()

  Renderiza la escena vista por la cámara y actualiza el lienzo.
- void **sync_all** ()

  Sincroniza todos los puertos.
- void **connect_all** ()

  Conecta todas las señales de los puertos.
- void slot_camera_id (int index)

  Respuesta a un cambio de la cámara seleccioinada en el puerto de cámara.
- void slot_camera_pos (double value)

  Respuesta a un cambio en la posición de la cámara.
- void slot_camera_rot (double value)

  Respuesta a un cambio en la rotación de la cámara.
- void slot_camera_center (bool checked)

  Respuesta a presionar el botón de orientar la cámara.
- void slot_camera_proj (bool checked)

  Respuesta a un cambio de la proyección en el puerto de vista.
- void slot_camera_skybox_id (int index)

  Respuesta a un cambio en el skybox seleccionado.
- void slot_object_id (int index)

  Respuesta a un cambio en el objeto seleccionado.
- void slot_object_pos (double value)

  Respuesta a un cambio en la posición del objeto.
- void slot_object_rot (double value)

  Respuesta a un cambio en la rotación del objeto.
- void slot_object_scale (double value)

  Respuesta a un cambio en el escalamiento del objeto.
- void slot_light_id (int index)

  Resouesta a un cambio de la luz seleccionada en el puerto de luces.
- void slot_light_state (Qt::CheckState state)

  Respuesta a un cambio del estado encendido en el puerto de luces.
- void slot_light_pos (double value)

  Respuesta a un cambio en la posición de la luz seleccionada.

- void slot_light_amb (double value)

  Respuesta a un cambio en la componente ambiental de la luz seleccionada.
- void slot_light_diff (double value)

  Respuesta a un cambio en la componente difusa de la luz seleccionada.
- void slot_light_spec (double value)

  Respuesta a un cambio en la componente especular de la luz seleccionada.
- void slot_material_id (int index)

  Respuesta a un cambio del material seleccionado.
- void slot_material_amb (double value)

  Respuesta a un cambio del coeficiente ambiental del material seleccionado.
- void slot_material_diff (double value)

  Respuesta a un cambio del coeficiente difuso del material seleccionado.
- void slot_material_spec (double value)

  Respuesta a un cambio del coeficiente especular del material seleccionado.
- void slot_material_sh (double value)

  Respuesta a un cambio en el brillo especular del material seleccionado.
- void slot_material_texture_id (int index)

  Respuesta a un cambio de la textura seleccionada.
- void slot_material_state (Qt::CheckState state)

  Respuesta a un cambio del estado de la propiedad reflexiva del material.

Atributos públicos

- Canvas **canvas**
- CameraDock **camera_dock**
- ObjectDock **object_dock**
- LightDock **light_dock**
- MaterialDock **material_dock**

4.21.1. Descripción detallada

Ventana principal de la aplicación.

4.21.2. Documentación de funciones miembro

4.21.2.1. slot_camera_center()

```
void Window::slot_camera_center (
            bool checked)
```

Respuesta a presionar el botón de orientar la cámara.

Parámetros

checked	el estado del botón si es activable

4.21.2.2. slot_camera_id()

```
void Window::slot_camera_id (
            int index)
```

Respuesta a un cambio de la cámara seleccioinada en el puerto de cámara.

Parámetros

index	el nuevo índice de la luz seleccionada

4.21.2.3. slot_camera_pos()

```
void Window::slot_camera_pos (
            double value)
```

Respuesta a un cambio en la posición de la cámara.

Parámetros

value	el nuevo valor de la posición

4.21.2.4. slot_camera_proj()

```
void Window::slot_camera_proj (
            bool checked)
```

Respuesta a un cambio de la proyección en el puerto de vista.

Parámetros

checked	el nuevo valor de la opción de proyección en perspectiva

4.21.2.5. slot_camera_rot()

```
void Window::slot_camera_rot (
            double value)
```

Respuesta a un cambio en la rotación de la cámara.

Parámetros

value	el nuevo valor de la rotación

4.21.2.6. slot_camera_skybox_id()

```
void Window::slot_camera_skybox_id (
            int index)
```

Respuesta a un cambio en el skybox seleccionado.

Parámetros

index	el nuevo índice del objeto seleccionado

4.21.2.7. slot_light_amb()

```
void Window::slot_light_amb (
            double value)
```

Respuesta a un cambio en la componente ambiental de la luz seleccionada.

Parámetros

value	el nuevo valor de la componente ambiental

4.21.2.8. slot_light_diff()

```
void Window::slot_light_diff (
            double value)
```

Respuesta a un cambio en la componente difusa de la luz seleccionada.

Parámetros

value	el nuevo valor de la componente difusa

4.21.2.9. slot_light_id()

```
void Window::slot_light_id (
            int index)
```

Resouesta a un cambio de la luz seleccionada en el puerto de luces.

Parámetros

index	el nuevo índice de la luz seleccionada

4.21.2.10. slot_light_pos()

```
void Window::slot_light_pos (
            double value)
```

Respuesta a un cambio en la posición de la luz seleccionada.

Parámetros

value	el nuevo valor de la posición

4.21.2.11. slot_light_spec()

```
void Window::slot_light_spec (
            double value)
```

Respuesta a un cambio en la componente especular de la luz seleccionada.

Parámetros

value	el nuevo valor de la componente especular

4.21.2.12. slot_light_state()

```
void Window::slot_light_state (
            Qt::CheckState state)
```

Respuesta a un cambio del estado encendido en el puerto de luces.

Parámetros

state	el nuevo valor del estado encendido

4.21.2.13. slot_material_amb()

```
void Window::slot_material_amb (
            double value)
```

Respuesta a un cambio del coeficiente ambiental del material seleccionado.

Parámetros

value	el nuevo del coeficiente ambiental

4.21.2.14. slot_material_diff()

```
void Window::slot_material_diff (
            double value)
```

Respuesta a un cambio del coeficiente difuso del material seleccionado.

Parámetros

value	el nuevo coeficiente difuso

4.21.2.15. slot_material_id()

```
void Window::slot_material_id (
            int index)
```

Respuesta a un cambio del material seleccionado.

Parámetros

index	el nuevo índice de l material seleccionado

4.21.2.16. slot_material_sh()

```
void Window::slot_material_sh (
            double value)
```

Respuesta a un cambio en el brillo especular del material seleccionado.

Parámetros

value	el nuevo valor del brillo

4.21.2.17. slot_material_spec()

```
void Window::slot_material_spec (
            double value)
```

Respuesta a un cambio del coeficiente especular del material seleccionado.

Parámetros

value	el nuevo coeficiente especular

4.21.2.18. slot_material_state()

```
void Window::slot_material_state (
            Qt::CheckState state)
```

Respuesta a un cambio del estado de la propiedad reflexiva del material.

Parámetros

state	el nuevo valor del estado de de la propiedad reflexiva

4.21.2.19. slot_material_texture_id()

```
void Window::slot_material_texture_id (
            int index)
```

Respuesta a un cambio de la textura seleccionada.

Parámetros

index	el nuevo índice de la textura seleccionada

4.21.2.20. slot_object_id()

```
void Window::slot_object_id (
            int index)
```

Respuesta a un cambio en el objeto seleccionado.

Parámetros

index	el nuevo índice del objeto seleccionado

4.21.2.21. slot_object_pos()

```
void Window::slot_object_pos (
            double value)
```

Respuesta a un cambio en la posición del objeto.

Parámetros

value	el nuevo valor de la posición

4.21.2.22. slot_object_rot()

```
void Window::slot_object_rot (
            double value)
```

Respuesta a un cambio en la rotación del objeto.

Parámetros

value	el nuevo valor de la rotación

4.21.2.23. slot_object_scale()

```
void Window::slot_object_scale (
            double value)
```

Respuesta a un cambio en el escalamiento del objeto.

Parámetros

value	el nuevo valor del escalamiento

La documentación de esta estructura está generada del siguiente archivo:

- include/gui.hpp

Capítulo 5

Documentación de archivos

5.1. Referencia del archivo include/app.hpp

```
#include <iostream>
#include <filesystem>
#include <QtWidgets>
#include "tiny_obj_loader.h"
#include "gmath.hpp"
#include "sampler.hpp"
```

Clases

- struct Vertex

  Vértice que se procesa en la línea de renderizado.
- struct Payload

  Carga que se transfiere entre etapas de la línea de renderizado.
- struct Light

  Propiedades de una luz puntual.
- struct Material

  Propiedades de un material.
- struct Transform

  Transformación de un objeto.
- struct CameraTransform

  Transformación de una cámara.

defines

- #define **WIDTH** 400
- #define **HEIGHT** 400
- #define **ASPECT_RATIO** ((double) WIDTH / HEIGHT)
- #define **FOV_V** (M_PI / 3)
- #define **EXT_V** 100
- #define **NEAR** 0.1
- #define **FAR** 1000
- #define **NUM_CAMERAS** 3
- #define **NUM_SKYBOXES** 5
- #define **NUM_OBJECTS** 7
- #define **NUM_LIGHTS** 2
- #define **NUM_MATERIALS** 2
- #define **NUM_TEXTURES** 6

Funciones

- bool load_obj (std::string file)

  Carga los vértices de un archivo obj al buffer de vértices.
- bool load_cube_map (std::string dir)

  Carga el skybox en una carpeta y actualiza el muestreador de skybos.
- bool load_texture (std::string file)

  Carga la textura del archivo especificado y actualiza el muestreador de textura.
- Payload skybox_vertex_shader (Vertex vertex, double ∗position)

  Genera la posición final de un vértice del skybox.
- void skybox_fragment_shader (Payload in, double ∗color)

  Genera el color final de un fragmento del skybox.
- Payload object_vertex_shader (Vertex vertex, double ∗position)

  Genera la posición final de un vértice del objeto.
- void object_fragment_shader (Payload in, double ∗color)

  Genera el color final de un fragmento del objeto.
- void **render_scene** ()

  Ejecuta la lógica para renderizar la escena.
- void **center_view** ()

  Actualiza la rotación de la cámara para que apunte hacia la posición del objeto actual.

Variables

- QImage **image**

  La imagen que se muestra en la ventana.
- Vertex **skybox_vertices** [6]

  Los vértices del skybox.
- std::vector< Vertex > **object_vertices**

  Los vértices del objeto.
- Transform **object_transforms** [NUM_OBJECTS]

  La transformación de los objetos.
- Material **materials** [NUM_MATERIALS]

  Los materiales.
- CameraTransform **camera_transforms** [NUM_CAMERAS]

  Las transformaciones de las cámaras.
- double **camera_ref** [3]

  El vector de referencia hacia arriba de las cámaras.
- double **model_mat** [4][4]

  La matriz de modelo.
- double **view_mat** [4][4]

  La matriz de vista.
- double **projection_mat** [4][4]

  La matriz de proyección.
- double **normal_mat** [4][4]

  La matriz normal.
- double **mvp_mat** [4][4]

  El producto de las matrices de proyección, vista y modelo.
- double **skybox_mat** [4][4]

  La matriz del skybox.
- void ∗ **mvp_chain** [3]

Arreglo con las matrices de proyección, vista y modelo.

- bool **perspective_flag**

  Bandera que indica si la proyección actual es en perspectiva.
- bool **reflective_material_flag**

  Bandera que indica si el material es reflexivo.
- Light **lights** [NUM_LIGHTS]

  Arreglo con las luces de la escena.
- double **global_ambient** [4]

  Componentes de la luz ambiental global de la escena.
- uint64_t **active_lights**

  Máscara de las luces encendidas.
- uint64_t **camera_id**

  Índice de la cámara seleccionada.
- uint64_t **skybox_id**

  Índice del skybox seleccionado.
- uint64_t **object_id**

  Índice del objeto seleccionado.
- uint64_t **light_id**

  Índice de la luz seleccionada.
- uint64_t **material_id**

  Índice del material seleccionado.
- uint64_t **texture_id**

  Índice de la textura seleccionada.
- uint32_t **color_buffer** [HEIGHT][WIDTH]

  Buffer de color que forma la imagen mostrada en el lienzo.
- double **depth_buffer** [HEIGHT][WIDTH]

  Buffer usado en la prueba de profundidad.
- QImage **texture_image**

  Imagen con la textura del material.
- Sampler **texture_sampler**

  Muestreador de la textura.
- QImage **cube_map_images** [6]

  Imágenes de las caras del skybox.
- std::string **skybox_dir_names** [NUM_SKYBOXES]

  Nombres de las carpetas con los skyboxes.
- SamplerCube **skybox_sampler**

  Muestreador del skybox.
- std::string **model_file_names** [NUM_OBJECTS]

  Nombres de los archivos con los modelos de los objetos.
- std::string **texture_file_names** [NUM_TEXTURES]

  Nombres de los archivos con las texturas.
- std::filesystem::path **texture_prefix**

  Prefijo para formar la ruta a una textura.
- std::filesystem::path **obj_prefix**

  Prefijo para formar la ruta a un modelo obj.

5.1.1. Documentación de funciones

5.1.1.1. load_cube_map()

```
bool load_cube_map (
            std::string dir)
```

Carga el skybox en una carpeta y actualiza el muestreador de skybos.

Parámetros

dir	es la carpeta donde se encuentran las imágenes de las caras

Devuelve

`true` si y solo si se pudo cargar el skybox con éxito

5.1.1.2. load_obj()

```
bool load_obj (
            std::string file)
```

Carga los vértices de un archivo obj al buffer de vértices.

Parámetros

file	es el archivo de donde se leen los vértices

Devuelve

`true` si y solo si se pudo cargar el modelo obj con éxito

5.1.1.3. load_texture()

```
bool load_texture (
            std::string file)
```

Carga la textura del archivo especificado y actualiza el muestreador de textura.

Parámetros

file	es el archivo con la textura

Devuelve

`true` si y solo si se pudo cargar la textura con éxito

5.1.1.4. object_fragment_shader()

```
void object_fragment_shader (
            Payload in,
            double * color)
```

Genera el color final de un fragmento del objeto.

Parámetros

in	es la carga que se recibe

color	es donde se guarda el color final

5.1.1.5. object_vertex_shader()

```
Payload object_vertex_shader (
            Vertex vertex,
            double * position)
```

Genera la posición final de un vértice del objeto.

Parámetros

vertex	es el vértice
position	es donde se guarda la posición final

Devuelve

la carga que se envía a los fragmentos

5.1.1.6. skybox_fragment_shader()

```
void skybox_fragment_shader (
            Payload in,
            double * color)
```

Genera el color final de un fragmento del skybox.

Parámetros

in	es la carga que se recibe
color	es donde se guarda el color final

5.1.1.7. skybox_vertex_shader()

```
Payload skybox_vertex_shader (
            Vertex vertex,
            double * position)
```

Genera la posición final de un vértice del skybox.

Parámetros

vertex	es el vértice
position	es donde se guarda la posición final

Devuelve

la carga que se envía a los fragmentos

5.2. app.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
00004
00005 # include <iostream>
00006 # include <filesystem>
00007 # include <QtWidgets>
00008
00009 # include "tiny_obj_loader.h"
00010 # include "gmath.hpp"
00011 # include "sampler.hpp"
00012
00013 # define WIDTH 400
00014 # define HEIGHT 400
00015 # define ASPECT_RATIO ((double) WIDTH / HEIGHT)
00016 # define FOV_V (M_PI / 3)
00017 # define EXT_V 100
00018 # define NEAR 0.1
00019 # define FAR 1000
00020 # define NUM_CAMERAS 3
00021 # define NUM_SKYBOXES 5
00022 # define NUM_OBJECTS 7
00023 # define NUM_LIGHTS 2
00024 # define NUM_MATERIALS 2
00025 # define NUM_TEXTURES 6
00026
00028 struct Vertex {
00029     double position [3];
00030     double normal [3];
00031     double texture [2];
00032 };
00033
00035 struct Payload {
00036     double view [3];
00037     double normal [3];
00038     double light [NUM_LIGHTS] [3];
00039     double texture [2];
00040 };
00041
00043 struct Light {
00044     double position [3];
00045     double ambient [4];
00046     double diffuse [4];
00047     double specular [4];
00048 };
00049
00051 struct Material {
00052     double ambient [4];
00053     double diffuse [4];
00054     double specular [4];
00055     double shininess;
00056 };
00057
00059 struct Transform {
00060     double position [3];
00061     double rotation [4];
00062     double scale [3];
00063 };
00064
00066 struct CameraTransform {
00067     double position [3];
00068     double rotation [4];
00069 };
00070
00072 extern QImage image;
00073
00075 extern Vertex skybox_vertices [6];
00076
00078 extern std::vector <Vertex> object_vertices;
00079
00081 extern Transform object_transforms [NUM_OBJECTS];
00082
00084 extern Material materials [NUM_MATERIALS];
00085
00087 extern CameraTransform camera_transforms [NUM_CAMERAS];
00088
00090 extern double camera_ref [3];
00091
00093 extern double model_mat [4] [4];
00094
00096 extern double view_mat [4] [4];
00097
```

```
extern double projection_mat [4] [4];

extern double normal_mat [4] [4];

extern double mvp_mat [4] [4];

extern double skybox_mat [4] [4];

extern void * mvp_chain [3];

extern bool perspective_flag;

extern bool reflective_material_flag;

extern Light lights [NUM_LIGHTS];

extern double global_ambient [4];

extern uint64_t active_lights;

extern uint64_t camera_id;

extern uint64_t skybox_id;

extern uint64_t object_id;

extern uint64_t light_id;

extern uint64_t material_id;

extern uint64_t texture_id;

extern uint32_t color_buffer [HEIGHT] [WIDTH];

extern double depth_buffer [HEIGHT] [WIDTH];

extern QImage texture_image;

extern Sampler texture_sampler;

extern QImage cube_map_images [6];

extern std::string skybox_dir_names [NUM_SKYBOXES];

extern SamplerCube skybox_sampler;

extern std::string model_file_names [NUM_OBJECTS];

extern std::string texture_file_names [NUM_TEXTURES];

extern std::filesystem::path texture_prefix;

extern std::filesystem::path obj_prefix;

bool load_obj (std::string file);

bool load_cube_map (std::string dir);

bool load_texture (std::string file);

Payload skybox_vertex_shader (Vertex vertex, double * position);

void skybox_fragment_shader (Payload in, double * color);

Payload object_vertex_shader (Vertex vertex, double * position);

void object_fragment_shader (Payload in, double * color);

void render_scene ();

void center_view ();
```

5.3. Referencia del archivo include/dip.hpp

```
#include <cstdint>
```

Clases

- struct Coord

Coordenadas de un pixel en una imagen.

- struct Rect

  Región rectangular de pixeles en una imagen.

- struct Extent

  Tamaño de una región rectangular en una imagen.

- struct Image

  Imagen con información del tamaño.

- struct Kernel

  Kernel simétrico para operaciones de convolución.

5.4. dip.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
00004
00005 # include <cstdint>
00006
00008 struct Coord {
00009     uint64_t x, y;
00010 };
00011
00013 struct Rect {
00014     Coord start, end;
00015 };
00016
00018 struct Extent {
00019     uint64_t width, height;
00020 };
00021
00023 struct Image {
00024     Extent extent;
00025     uint64_t channels;
00026     void * data;
00027 };
00028
00030 struct Kernel {
00031     uint64_t radius;
00032     void * data;
00033 };
```

5.5. Referencia del archivo include/gmath.hpp

```
#include <cstdint>
#include <cstring>
#include <cmath>
```

Funciones

- double clamp (double x, double a, double b)

  Limita un valor `x` a un intervalo `[a,b]`.

- double lerp (double t, double a, double b)

  Realiza una interpolación lineal para `t`.

- double dlerp (double t, double a, double b)

  Realiza una interpolación lineal para `t`.

- double blerp (double u, double v, double a, double b, double c)

Realiza una interpolación lineal para `u` y `v`.

- double frac_part (double x)

  Regresa la parte fraccionaria no negativa de `x`.
- double multilinear_interpolation (uint64_t n, double ∗target, double(∗inputs)[2], double ∗outputs)

  Realiza una interpolación multilineal en `n` dimensiones.
- double vec_norm (double ∗vec, uint64_t n)

  Calcula la norma de un vector.
- void vec_unit (double ∗vec_a, double ∗vec_b, uint64_t n)

  Construye el vector unitario en la dirección del vector `vec`.
- void vec_to_homogeneous (double ∗vec_a, double ∗vec_b, uint64_t n)

  Convierte un vector en coordenadas normales a coordenadas homogéneas.
- void vec_from_homogeneous (double ∗vec_a, double ∗vec_b, uint64_t n)

  Convierte un vector en coordenadas homogeneas a coordenadas normales.
- void vec_neg (double ∗vec_a, double ∗vec_b, uint64_t n)

  Construye la negación de un vector.
- void vec_add (double ∗vec_a, double ∗vec_b, double ∗vec_c, uint64_t n)

  Suma dos vectores.
- void vec_sub (double ∗vec_a, double ∗vec_b, double ∗vec_c, uint64_t n)

  Resta dos vectores.
- void vec_emul (double ∗vec_a, double ∗vec_b, double ∗vec_c, uint64_t n)

  Multiplica dos vectores elemento por elemento.
- void vec_scalar_mul (double ∗vec_a, double scalar, double ∗vec_b, uint64_t n)

  Multiplica un vector por un escalar.
- double vec_dot_prod (double ∗vec_a, double ∗vec_b, uint64_t n)

  Calcula el producto punto entre dos vectores.
- void vec3_cross_prod (double ∗vec_a, double ∗vec_b, double ∗vec_c)

  Calcula el producto cruz entre vectores de 3 dimensiones.
- void vec_reflect (double ∗incident, double ∗normal, double ∗reflected, uint64_t n)

  Construye el vector unitario reflejado.
- void mat_identity (void ∗mat, uint64_t n)

  Construye la matriz identidad.
- void mat_transpose (void ∗mat_a, void ∗mat_b, uint64_t n_1, uint64_t n_2)

  Construye la transpuesta de una matriz.
- void mat_add (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2)

  Suma dos matrices.
- void mat_sub (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2)

  Resta dos matrices.
- void mat_emul (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2)

  Multiplica dos matrices elemento por elemento.
- void mat_scalar_mul (void ∗mat_a, double scalar, void ∗mat_b, uint64_t n_1, uint64_t n_2)

  Multiplica una matriz por un escalar.
- void mat_vec_mul (void ∗mat, double ∗vec_a, double ∗vec_b, uint64_t n_1, uint64_t n_2)

  Multiplica una matriz por un vector columna.
- void mat_mul (void ∗mat_a, void ∗mat_b, void ∗mat_c, uint64_t n_1, uint64_t n_2, uint64_t n_3)

  Multiplica dos matrices.
- double mat_inverse (void ∗mat_a, void ∗mat_b, uint64_t n)

  Construye la inversa de una matriz. Se pierde el contenido de `mat_a`.
- void camera_look_at_mat4 (double ∗position, double ∗target, double ∗ref, void ∗mat)

  Construye la matriz de vista para una cámara en `position` viendo hacia `target` con vector de referencia hacia arriba `ref`.
- void orthogonal_proj_mat4 (double aspect_ratio, double ext_v, double near, double far, void ∗mat)

Construye la matriz de proyección ortogonal.

- void inv_orthogonal_proj_mat4 (double aspect_ratio, double ext_v, double near, double far, void ∗mat)

  Construye la inversa de la matriz de proyección ortogonal.
- void perspective_proj_mat4 (double aspect_ratio, double fov_v, double near, double far, void ∗mat)

  Construye la matriz de proyección en perspectiva.
- void inv_perspective_proj_mat4 (double aspect_ratio, double fov_v, double near, double far, void ∗mat)

  Construye la inversa de la matriz de proyección en perspectiva.
- void transform_euler_zyx_mat4 (double ∗position, double ∗rotation, double ∗scale, void ∗mat)

  Construye la matriz de transformación a partir del desplazamiento, rotación y escalamiento. Se considera que la rotación está dada por ángulos de euler ZYX. El orden de la transformación es: escalamiento, rotación alrededor de Z, rotación alrededor de Y, rotación alrededor de X y traslación.
- void camera_euler_zyx_mat4 (double ∗position, double ∗rotation, void ∗mat)

  Construye la matriz de vista a partir de la transformación de la cámara.
- void mat4_to_euler_zyx (void ∗mat, double ∗rotation)

  Extrae los ángulos de euler ZYX de una matriz de rotación.
- void mat4_chain_mul (void ∗∗array, uint64_t n, void ∗mat)

  Multiplica una secuencia de matrices 4 4.
- void bound_points (void ∗mat, void ∗aabb, uint64_t n_1, uint64_t n_2)

  Calcula la caja delimitadora alineada con los ejes mínima para los puntos en las filas de la matriz `mat`*.*
- void linear_param (void ∗mat_a, void ∗mat_b, uint64_t n_1, uint64_t n_2)

  Construye una parametrización lineal para los puntos en las filas de la matriz `mat_a`*.*
- bool aabb_empty (void ∗aabb, uint64_t n)

  Determina si una caja delimitadora alineada con los ejes es vacía.
- void aabb_intersection (void ∗aabb_a, void ∗aabb_b, void ∗aabb_c, uint64_t n)

  Intersecta dos cajas delimitadoras alineadas con los ejes.
- void line_normal (double ∗point_a, double ∗point_b, double ∗normal)

  Calcula la normal de la recta que pasa por dos puntos en el plano.
- void plane_normal (double ∗point_a, double ∗point_b, double ∗point_c, double ∗normal)

  Calcula la normal del plano que pasa por tres puntos en el espacio.

5.5.1. Documentación de funciones

5.5.1.1. aabb_empty()

```
bool aabb_empty (
            void * aabb,
            uint64_t n)
```

Determina si una caja delimitadora alineada con los ejes es vacía.

Parámetros

aabb	es el caja
n	es la dimensión de `aabb`

Devuelve

`true` si y solo si la caja está vacía

5.5.1.2. aabb_intersection()

```
void aabb_intersection (
            void * aabb_a,
            void * aabb_b,
            void * aabb_c,
            uint64_t n)
```

Intersecta dos cajas delimitadoras alineadas con los ejes.

Parámetros

aabb↩ _a	es la primer caja
aabb↩ _b	es la segunda caja
aabb↩ _c	es donde se guarda el resultado
n	es la dimensión de `aabb_a` y `aabb_b`

5.5.1.3. blerp()

```
double blerp (
            double u,
            double v,
            double a,
            double b,
            double c)
```

Realiza una interpolación lineal para `u` y `v`.

Parámetros

u	es el primer valor de interpolación
v	es el segundo valor de interpolación
a	es el resultado si `u = 0` y `v = 0`
b	es el resultado si `u = 1` y `v = 0`
c	es el resultado si `u = 0` y `v = 1`

Devuelve

la interpolación lineal para `u` y `v`

5.5.1.4. bound_points()

```
void bound_points (
            void * mat,
            void * aabb,
            uint64_t n_1,
            uint64_t n_2)
```

Calcula la caja delimitadora alineada con los ejes mínima para los puntos en las filas de la matriz `mat`.

Parámetros

mat	es la matriz con los puntos

aabb	es donde se guarda el resultado
n_1	es el número de filas de `mat`
n_2	es el número de columnas de `mat`

5.5.1.5. camera_euler_zyx_mat4()

```
void camera_euler_zyx_mat4 (
            double * position,
            double * rotation,
            void * mat)
```

Construye la matriz de vista a partir de la transformación de la cámara.

Parámetros

position	es la posición de la cámara
rotation	es la rotación de la cámara en ángulos de euler ZYX
mat	es donde se guarda el resultado

5.5.1.6. camera_look_at_mat4()

```
void camera_look_at_mat4 (
            double * position,
            double * target,
            double * ref,
            void * mat)
```

Construye la matriz de vista para una cámara en `position` viendo hacia `target` con vector de referencia hacia arriba `ref`.

Parámetros

position	es la posición de la cámara
target	es la posición adonde apunta la cámara
ref	es el vector de referencia hacia arriba
mat	es donde se guarda el resultado

5.5.1.7. clamp()

```
double clamp (
            double x,
            double a,
            double b)
```

Limita un valor `x` a un intervalo `[a,b]`.

Parámetros

x	es el valor

a	es el límite inferior del intervalo
b	es el límite superior del intervalo

Devuelve

`x` si se encuentra en `[a,b]` o el límite más cercano a `x` si no está en el intervalo

5.5.1.8. dlerp()

```
double dlerp (
            double t,
            double a,
            double b)
```

Realiza una interpolación lineal para `t`.

Parámetros

t	es el valor de interpolación
a	es el resultado si `t = -1`
b	es el resultado si `t = 1`

Devuelve

la interpolación lineal para `t`

5.5.1.9. frac_part()

```
double frac_part (
            double x)
```

Regresa la parte fraccionaria no negativa de `x`.

Parámetros

x	es el valor

Devuelve

la parte fraccionaria no negativa de `x`

5.5.1.10. inv_orthogonal_proj_mat4()

```
void inv_orthogonal_proj_mat4 (
            double aspect_ratio,
            double ext_v,
            double near,
            double far,
            void * mat)
```

Construye la inversa de la matriz de proyección ortogonal.

Parámetros

aspect_ratio	es la razón de aspecto del volumen de visión

ext_v	es la extensión vertical del volumen de visión
near	es la distancia al plano cercano
far	es la distancia al plano lejano
mat	es donde se guarda el resultado

5.5.1.11. inv_perspective_proj_mat4()

```
void inv_perspective_proj_mat4 (
            double aspect_ratio,
            double fov_v,
            double near,
            double far,
            void * mat)
```

Construye la inversa de la matriz de proyección en perspectiva.

Parámetros

aspect_ratio	es la razón de aspecto del volumen de visión
fov_v	es el campo de visión vertical
near	es la distancia al plano cercano
far	es la distancia al plano lejano
mat	es donde se guarda el resultado

5.5.1.12. lerp()

```
double lerp (
            double t,
            double a,
            double b)
```

Realiza una interpolación lineal para `t`.

Parámetros

t	es el valor de interpolación
a	es el resultado si `t = 0`
b	es el resultado si `t = 1`

Devuelve

la interpolación lineal para `t`

5.5.1.13. line_normal()

```
void line_normal (
            double * point_a,
            double * point_b,
            double * normal)
```

Calcula la normal de la recta que pasa por dos puntos en el plano.

Parámetros

point↩ _a	es el primer punto
point↩ _b	es el segundo punto
normal	es donde se guarda el resultado

5.5.1.14. linear_param()

```
void linear_param (
            void * mat_a,
            void * mat_b,
            uint64_t n_1,
            uint64_t n_2)
```

Construye una parametrización lineal para los puntos en las filas de la matriz `mat_a`.

Parámetros

mat↩ _a	es la matriz con los puntos
mat↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a`

5.5.1.15. mat4_chain_mul()

```
void mat4_chain_mul (
            void ** array,
            uint64_t n,
            void * mat)
```

Multiplica una secuencia de matrices 4 4.

Parámetros

array	es el arreglo con las matrices

n	es el tamaño del arreglo
mat	es donde se guarda el resultado

5.5.1.16. mat4_to_euler_zyx()

```
void mat4_to_euler_zyx (
            void * mat,
            double * rotation)
```

Extrae los ángulos de euler ZYX de una matriz de rotación.

Parámetros

mat	es la matriz de rotación
rotation	es donde se guardan los ángulos de euler ZYX

5.5.1.17. mat_add()

```
void mat_add (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2)
```

Suma dos matrices.

Parámetros

mat↩ _a	es la primer matriz
mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a` y `mat_b`
n_2	es el número de columnas de `mat_a`y `mat_b`

5.5.1.18. mat_emul()

```
void mat_emul (
            void * mat_a,
            void * mat_b,
            void * mat_c,
```

```
uint64_t n_1,
uint64_t n_2)
```

Multiplica dos matrices elemento por elemento.

Parámetros

mat↩ _a	es la primer matriz

mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a` y `mat_b`
n_2	es el número de columnas de `mat_a`y `mat_b`

5.5.1.19. mat_identity()

```
void mat_identity (
            void * mat,
            uint64_t n)
```

Construye la matriz identidad.

Parámetros

mat	es donde se guarda el resultado
n	es el número de filas y columnas de la matriz

5.5.1.20. mat_inverse()

```
double mat_inverse (
            void * mat_a,
            void * mat_b,
            uint64_t n)
```

Construye la inversa de una matriz. Se pierde el contenido de `mat_a`.

Parámetros

mat↩ _a	es la matriz
mat↩ _b	es donde se guarda el resultado
n	es el número de filas y columnas de `mat_a`

Devuelve

el determinante de `mat_a`

5.5.1.21. mat_mul()

```
void mat_mul (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2,
            uint64_t n_3)
```

Multiplica dos matrices.

Parámetros

mat↩ _a	es la primer matriz
mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a` y el número de filas de `mat_b`
n_3	es el número de columnas de `mat_b`

5.5.1.22. mat_scalar_mul()

```
void mat_scalar_mul (
            void * mat_a,
            double scalar,
            void * mat_b,
            uint64_t n_1,
            uint64_t n_2)
```

Multiplica una matriz por un escalar.

Parámetros

mat↩ _a	es la matriz
scalar	es el escalar
mat↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a`

5.5.1.23. mat_sub()

```
void mat_sub (
            void * mat_a,
            void * mat_b,
            void * mat_c,
            uint64_t n_1,
            uint64_t n_2)
```

Resta dos matrices.

Parámetros

mat↩ _a	es la primer matriz
mat↩ _b	es la segunda matriz
mat↩ _c	es donde se guarda el resultado
n_1	es el número de filas de `mat_a` y `mat_b`
n_2	es el número de columnas de `mat_a`y `mat_b`

5.5.1.24. mat_transpose()

```
void mat_transpose (
            void * mat_a,
            void * mat_b,
            uint64_t n_1,
            uint64_t n_2)
```

Construye la transpuesta de una matriz.

Parámetros

mat↩ _a	es la matriz
mat↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat_a`
n_2	es el número de columnas de `mat_a`

5.5.1.25. mat_vec_mul()

```
void mat_vec_mul (
            void * mat,
```

```
            double * vec_a,
            double * vec_b,
            uint64_t n_1,
            uint64_t n_2)
```

Multiplica una matriz por un vector columna.

Parámetros

mat	es la matriz
vec↩ _a	es el vector columna
vec↩ _b	es donde se guarda el resultado
n_1	es el número de filas de `mat`
n_2	es el número de columnas de `mat` y `vec_a`

5.5.1.26. multilinear_interpolation()

```
double multilinear_interpolation (
            uint64_t n,
            double * target,
            double(*) inputs[2],
            double * outputs)
```

Realiza una interpolación multilineal en `n` dimensiones.

Parámetros

n	es el número de dimensioines
target	es el valor de interpolación
inputs	es un arreglo con los pares de valores de cada variable de entrada
outputs	es un arreglo con los valores de la variable de salida correspondientes a cada combinación de los pares de entradas

Devuelve

la interpolación multilineal para `target`

5.5.1.27. orthogonal_proj_mat4()

```
void orthogonal_proj_mat4 (
            double aspect_ratio,
            double ext_v,
            double near,
```

```
double far,
void * mat)
```

Construye la matriz de proyección ortogonal.

Parámetros

aspect_ratio	es la razón de aspecto del volumen de visión

ext_v	es la extensión vertical del volumen de visión
near	es la distancia al plano cercano
far	es la distancia al plano lejano
mat	es donde se guarda el resultado

5.5.1.28. perspective_proj_mat4()

```
void perspective_proj_mat4 (
            double aspect_ratio,
            double fov_v,
            double near,
            double far,
            void * mat)
```

Construye la matriz de proyección en perspectiva.

Parámetros

aspect_ratio	es la razón de aspecto del volumen de visión
fov_v	es el campo de visión vertical
near	es la distancia al plano cercano
far	es la distancia al plano lejano
mat	es donde se guarda el resultado

5.5.1.29. plane_normal()

```
void plane_normal (
            double * point_a,
            double * point_b,
            double * point_c,
            double * normal)
```

Calcula la normal del plano que pasa por tres puntos en el espacio.

Parámetros

point↩ _a	es el primer punto
point↩ _b	es el segundo punto
point↩ _c	es el tercer punto
normal	es donde se guarda el resultado

5.5.1.30. transform_euler_zyx_mat4()

```
void transform_euler_zyx_mat4 (
            double * position,
            double * rotation,
            double * scale,
            void * mat)
```

Construye la matriz de transformación a partir del desplazamiento, rotación y escalamiento. Se considera que la rotación está dada por ángulos de euler ZYX. El orden de la transformación es: escalamiento, rotación alrededor de Z, rotación alrededor de Y, rotación alrededor de X y traslación.

Parámetros

position	es el desplazamiento
rotation	es la rotación en ángulos de euler ZYX
scale	es el escalamiento
mat	es donde se guarda el resultado

5.5.1.31. vec3_cross_prod()

```
void vec3_cross_prod (
            double * vec_a,
            double * vec_b,
            double * vec_c)
```

Calcula el producto cruz entre vectores de 3 dimensiones.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado

5.5.1.32. vec_add()

```
void vec_add (
            double * vec_a,
            double * vec_b,
            double * vec_c,
            uint64_t n)
```

Suma dos vectores.

Parámetros

vec↩ _a	es el primer vector

vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado
n	es la dimensión de `vec_a` y `vec_b`

5.5.1.33. vec_dot_prod()

```
double vec_dot_prod (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Calcula el producto punto entre dos vectores.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
n	es la dimensión `vec_a` y `vec_b`

Devuelve

el producto punto de ambos vectores

5.5.1.34. vec_emul()

```
void vec_emul (
            double * vec_a,
            double * vec_b,
            double * vec_c,
            uint64_t n)
```

Multiplica dos vectores elemento por elemento.

Parámetros

vec↩ _a	es el primer vector
vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado
n	es la dimensión de `vec_a` y `vec_b`

5.5.1.35. vec_from_homogeneous()

```
void vec_from_homogeneous (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Convierte un vector en coordenadas homogeneas a coordenadas normales.

Parámetros

vec↩ _a	es el vector en coordenadas homogéneas
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.36. vec_neg()

```
void vec_neg (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Construye la negación de un vector.

Parámetros

vec↩ _a	es el vector
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.37. vec_norm()

```
double vec_norm (
            double * vec,
            uint64_t n)
```

Calcula la norma de un vector.

Parámetros

vec	es el vector
n	es la dimensión de `vec`

Devuelve

la norma de `vec`

5.5.1.38. vec_reflect()

```
void vec_reflect (
            double * incident,
            double * normal,
            double * reflected,
            uint64_t n)
```

Construye el vector unitario reflejado.

Parámetros

incident	es el vector unitario incidente
normal	es el vector unitario de la normal
reflected	es donde se guarda el resultado
n	es la dimensión de `incident` y `normal`

5.5.1.39. vec_scalar_mul()

```
void vec_scalar_mul (
            double * vec_a,
            double scalar,
            double * vec_b,
            uint64_t n)
```

Multiplica un vector por un escalar.

Parámetros

vec↩ _a	es el vector
scalar	es el escalar
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.40. vec_sub()

```
void vec_sub (
            double * vec_a,
            double * vec_b,
            double * vec_c,
            uint64_t n)
```

Resta dos vectores.

Parámetros

vec↩ _a	es el primer vector

vec↩ _b	es el segundo vector
vec↩ _c	es donde se guarda el resultado
n	es la dimensión de `vec_a` y `vec_b`

5.5.1.41. vec_to_homogeneous()

```
void vec_to_homogeneous (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Convierte un vector en coordenadas normales a coordenadas homogéneas.

Parámetros

vec↩ _a	es el vector en coordenadas normales
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.5.1.42. vec_unit()

```
void vec_unit (
            double * vec_a,
            double * vec_b,
            uint64_t n)
```

Construye el vector unitario en la dirección del vector `vec`.

Parámetros

vec↩ _a	es el vector
vec↩ _b	es donde se guarda el resultado
n	es la dimensión de `vec_a`

5.6. gmath.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
```

```
# include <cstdint>
# include <cstring>
# include <cmath>

double clamp (double x, double a, double b);

double lerp (double t, double a, double b);

double dlerp (double t, double a, double b);

double blerp (double u, double v, double a, double b, double c);

double frac_part (double x);

double multilinear_interpolation (uint64_t n, double * target, double (* inputs) [2], double *
outputs);

double vec_norm (double * vec, uint64_t n);

void vec_unit (double * vec_a, double * vec_b, uint64_t n);

void vec_to_homogeneous (double * vec_a, double * vec_b, uint64_t n);

void vec_from_homogeneous (double * vec_a, double * vec_b, uint64_t n);

void vec_neg (double * vec_a, double * vec_b, uint64_t n);

void vec_add (double * vec_a, double * vec_b, double * vec_c, uint64_t n);

void vec_sub (double * vec_a, double * vec_b, double * vec_c, uint64_t n);

void vec_emul (double * vec_a, double * vec_b, double * vec_c, uint64_t n);

void vec_scalar_mul (double * vec_a, double scalar, double * vec_b, uint64_t n);

double vec_dot_prod (double * vec_a, double * vec_b, uint64_t n);

void vec3_cross_prod (double * vec_a, double * vec_b, double * vec_c);

void vec_reflect (double * incident, double * normal, double * reflected, uint64_t n);

void mat_identity (void * mat, uint64_t n);

void mat_transpose (void * mat_a, void * mat_b, uint64_t n_1, uint64_t n_2);

void mat_add (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2);

void mat_sub (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2);

void mat_emul (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2);

void mat_scalar_mul (void * mat_a, double scalar, void * mat_b, uint64_t n_1, uint64_t n_2);

void mat_vec_mul (void * mat, double * vec_a, double * vec_b, uint64_t n_1, uint64_t n_2);

void mat_mul (void * mat_a, void * mat_b, void * mat_c, uint64_t n_1, uint64_t n_2, uint64_t n_3);

double mat_inverse (void * mat_a, void * mat_b, uint64_t n);

void camera_look_at_mat4 (double * position, double * target, double * ref, void * mat);

void orthogonal_proj_mat4 (double aspect_ratio, double ext_v, double near, double far, void * mat);

void inv_orthogonal_proj_mat4 (double aspect_ratio, double ext_v, double near, double far, void *
mat);

void perspective_proj_mat4 (double aspect_ratio, double fov_v, double near, double far, void * mat);

void inv_perspective_proj_mat4 (double aspect_ratio, double fov_v, double near, double far, void *
mat);

void transform_euler_zyx_mat4 (double * position, double * rotation, double * scale, void * mat);

void camera_euler_zyx_mat4 (double * position, double * rotation, void * mat);

void mat4_to_euler_zyx (void * mat, double * rotation);

void mat4_chain_mul (void * * array, uint64_t n, void * mat);

void bound_points (void * mat, void * aabb, uint64_t n_1, uint64_t n_2);

void linear_param (void * mat_a, void * mat_b, uint64_t n_1, uint64_t n_2);

bool aabb_empty (void * aabb, uint64_t n);

void aabb_intersection (void * aabb_a, void * aabb_b, void * aabb_c, uint64_t n);
```

```
00289
00294 void line_normal (double * point_a, double * point_b, double * normal);
00295
00301 void plane_normal (double * point_a, double * point_b, double * point_c, double * normal);
```

5.7. Referencia del archivo include/gui.hpp

```
#include "pipeline.hpp"
```

Clases

- struct Canvas

  Lienzo donde se muestra la imagen en la ventana.
- struct CameraDock

  Puerto para el control de la cámara.
- struct ObjectDock

  Puerto para el control del objeto.
- struct LightDock

  Puerto para el control de las luces.
- struct MaterialDock

  Puerto para el control del material.
- struct Window

  Ventana principal de la aplicación.

5.8. gui.hpp

Ir a la documentación de este archivo.

```
00001
00002
00003 # pragma once
00004
00005 # include "pipeline.hpp"
00006
00008 struct Canvas : public QWidget {
00009
00012     void paintEvent(QPaintEvent * event);
00013
00014 };
00015
00017 struct CameraDock : public QDockWidget {
00018
00019     QWidget widget;
00020     QVBoxLayout layout;
00021     QComboBox camera_dropdown;
00022     QLabel transform_label;
00023     QWidget group;
00024     QGridLayout group_layout;
00025     QLabel row_labels [2];
00026     QLabel col_labels [3];
00027     QDoubleSpinBox pos_boxes [3];
00028     QDoubleSpinBox rot_boxes [3];
00029     QPushButton button;
00030     QLabel proj_label;
00031     QRadioButton proj_buttons [2];
00032     QLabel skybox_label;
00033     QComboBox skybox_dropdown;
00034
00036     CameraDock ();
00037
00039     void syncValues ();
00040
```

```
};

struct ObjectDock : public QDockWidget {

    QWidget widget;
    QVBoxLayout layout;
    QComboBox dropdown;
    QLabel label;
    QWidget group;
    QGridLayout group_layout;
    QLabel row_labels [3];
    QLabel col_labels [3];
    QDoubleSpinBox pos_boxes [3];
    QDoubleSpinBox rot_boxes [3];
    QDoubleSpinBox scale_boxes [3];


    ObjectDock ();

    void syncValues ();

};

struct LightDock : public QDockWidget {

    QWidget widget;
    QVBoxLayout layout;
    QComboBox dropdown;
    QCheckBox checkbox;
    QLabel transform_label;
    QWidget transform_group;
    QGridLayout transform_group_layout;
    QLabel transform_row_label;
    QLabel transform_col_labels [3];
    QDoubleSpinBox pos_boxes [3];
    QLabel component_label;
    QWidget component_group;
    QGridLayout component_group_layout;
    QLabel component_row_labels [3];
    QLabel component_col_labels [4];
    QDoubleSpinBox amb_boxes [4];
    QDoubleSpinBox diff_boxes [4];
    QDoubleSpinBox spec_boxes [4];

    LightDock ();

    void syncValues ();

};

struct MaterialDock : public QDockWidget {

    QWidget widget;
    QVBoxLayout layout;
    QComboBox material_dropdown;
    QLabel coef_label;
    QWidget group;
    QGridLayout group_layout;
    QLabel row_labels [3];
    QLabel col_labels [4];
    QDoubleSpinBox amb_boxes [4];
    QDoubleSpinBox diff_boxes [4];
    QDoubleSpinBox spec_boxes [4];
    QLabel sh_label;
    QDoubleSpinBox sh_box;
    QLabel texture_label;
    QComboBox texture_dropdown;
    QCheckBox checkbox;

    MaterialDock ();

    void syncValues ();

};

struct Window : public QMainWindow {

    Canvas canvas;
    CameraDock camera_dock;
    ObjectDock object_dock;
    LightDock light_dock;
    MaterialDock material_dock;

    Window ();

    void draw ();

```

```
    void sync_all ();

    void connect_all ();

    void slot_camera_id (int index);

    void slot_camera_pos (double value);

    void slot_camera_rot (double value);

    void slot_camera_center (bool checked);

    void slot_camera_proj (bool checked);

    void slot_camera_skybox_id (int index);

    void slot_object_id (int index);

    void slot_object_pos (double value);

    void slot_object_rot (double value);

    void slot_object_scale (double value);

    void slot_light_id (int index);

    void slot_light_state (Qt::CheckState state);

    void slot_light_pos (double value);

    void slot_light_amb (double value);

    void slot_light_diff (double value);

    void slot_light_spec (double value);

    void slot_material_id (int index);

    void slot_material_amb (double value);

    void slot_material_diff (double value);

    void slot_material_spec (double value);

    void slot_material_sh (double value);

    void slot_material_texture_id (int index);

    void slot_material_state (Qt::CheckState state);

};
```

5.9. Referencia del archivo include/pipeline.hpp

```
#include "app.hpp"
```

Clases

- struct Fragment

  Fragmento generado por el rasterizador.
- struct Pipeline

  Línea de renderizado por rasterización.

Variables

- Pipeline **pipeline**

  Línea de renderizado de la aplicación.

5.10. pipeline.hpp

Ir a la documentación de este archivo.

```


# pragma once

# include "app.hpp"

struct Fragment {
    uint64_t x, y;
    double u, v, depth;
    double color [4];
};

struct Pipeline {

    uint64_t num_primitives, width, height, primitive_id, vertex_id;
    Fragment fragment;
    Payload out [3];
    Payload in;
    uint64_t viewport [2] [2];
    uint64_t raster_rect [2] [2];
    double homogeneous_coordinates [3];
    double positions_3d [3] [3];
    double boundary_3d [2] [3];
    double positions_2d [3] [2];
    double boundary_2d [2] [2];
    void * color_buffer;
    void * depth_buffer;
    void * vertex_buffer;
    Payload (* vertex_shader) (Vertex, double *);
    void (* fragment_shader) (Payload, double *);
    bool front_face_clockwise;
    bool invert_y_viewport;
    bool depth_write_disabled;

    void clear_buffers ();

    void draw ();

    bool clipping_test ();

    void to_plane ();

    void to_viewport ();

    void rasterize ();

    bool depth_test ();

    void map_barycentric_coordinates ();

    void build_payload ();

    void blend_fragment ();

};

extern Pipeline pipeline;
```

5.11. sampler.hpp

```


# pragma once

# include "gmath.hpp"
# include "dip.hpp"

enum SamplerFilter : uint64_t {
    SAMPLER_FILTER_NEAREST,
    SAMPLER_FILTER_LINEAR
};

enum SamplerAddressMode : uint64_t {
    SAMPLER_ADDRESS_MODE_CLAMP,
    SAMPLER_ADDRESS_MODE_REPEAT
};

```

```
enum SamplerCubeFace : uint64_t {
    SAMPLER_CUBE_FACE_LEFT,
    SAMPLER_CUBE_FACE_RIGHT,
    SAMPLER_CUBE_FACE_BACK,
    SAMPLER_CUBE_FACE_FRONT,
    SAMPLER_CUBE_FACE_DOWN,
    SAMPLER_CUBE_FACE_UP
};

struct Sampler {

    Image image;
    SamplerFilter filter;
    SamplerAddressMode address_mode;

    void raw_sample_nearest (double * position, double * sample);

    void raw_sample_bilinear (double * position, double * sample);

    void sample (double * position, double * sample);

};

struct SamplerCube {

    Image cube_map [6];
    SamplerFilter filter;

    void sample (double * direction, double * sample);

};
```